The League: self-improvement against the live top 10

Architecture, commands and debugging guide

Patient Wolves — Battlecode 2026a, branch `league`

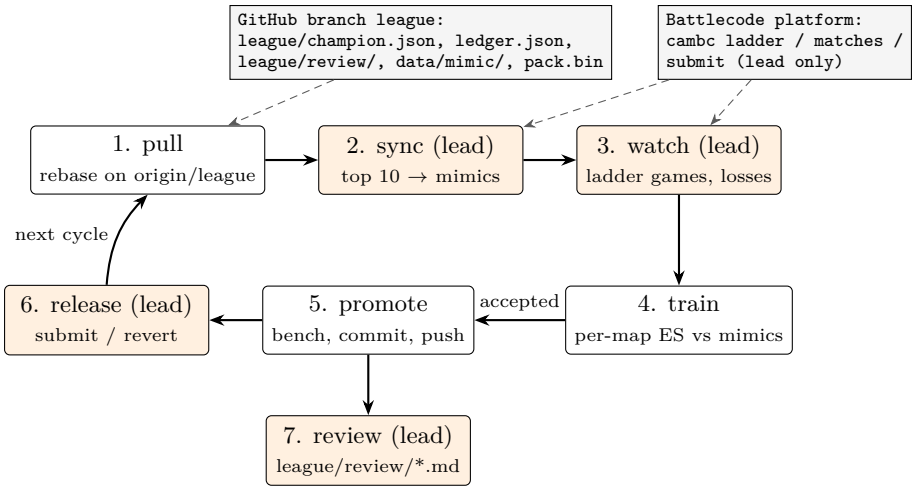
October 2026

1 What it does, in one paragraph

A single daemon, `harness/league.py`, runs on every teammate's PC. It reads the platform ladder and keeps a *mimic* of each top-10 team: our own engine (`bots/main`) driven by that team's macro (draft, emblem builds, opening routes, tribute and shop cadence), extracted from their recent ladder games. It also mimics teams we recently lost to. Our bot's trainable vector θ (102 floats, one copy per map) is improved by evolution strategies (ES) against these mimics, endlessly. When a candidate beats the champion, it is promoted, benchmarked against every mimic, committed and pushed to branch `league`, so every PC trains from the newest weights. One *lead* PC also watches our ladder games. A loss queues its map for training, and with `--auto-submit` a better champion is uploaded after a loss or a clear benchmark gain. Whatever ES cannot fix within θ is written up for humans in a review report, which separates *the mimic is wrong* from *the model is missing a feature*.

2 Architecture

2.1 The cycle



Orange steps run only on the lead PC (`--lead`). A worker PC runs steps 1, 4 and 5. Each cycle takes a few minutes: about 90 games on one map, plus about 340 benchmark games after a promotion.

1. **pull** (`integrate`): `git fetch` and `rebase` on `origin/league`. If the code or the champion changed upstream, `bots/main/src/pack/pack.bin` is regenerated from `league/champion.json` (`apply_champion`). `params.check()` stops the daemon if `learned.rs` and the registry disagree.
2. **sync** (`sync`, every 3 h): `cambc ladder` gives the top 10 (excluding us), plus up to three teams we recently lost to. For each, `download_games` fetches the 10 newest finished games, then `mimic.extract` and `mimic.pack_spec` write `data/mimic/<slug>.spec.json`. The ledger gets a rank and rating snapshot and the new mimic *pool*.
3. **watch** (`watch`, every 10 min): our new ladder games go into `ledger.ladder`, each tagged with the version that was live at the time. Every loss joins `ledger.losses` with its death summary (deaths by context: camp, champion, ...).

4. **train** (`train`): picks a map (the oldest open loss, otherwise the map we do worst on in the latest benchmark, otherwise round-robin). It samples $\lambda = 8$ antithetic candidates around that map's θ and plays them, together with the champion, against 4 PFSP-chosen mimics and 1 anchor bot, on that map only, both sides, in one parallel batch.
5. **promote**: the gate in §3 decides. On acceptance: update `champion.json` and `pack.bin`, run a release build check, benchmark against every mimic on every map, commit and push.
6. **release** (`release_decision`, `revert_decision`): see §3.
7. **review** (`write_review`): after each sync or watch, see §4.

2.2 Components

`harness/league.py`	The loop and its subcommands. Pure, unit-tested parts: `merge_champion`, `merge_ledger`, `release_decision`, `revert_decision`, `splice_notes` (`tests/test_league.py`).
`harness/mimic.py`	Team macro clone. *Cloned:* draft per team size, emblem build order per champion kind, first 4 camp groups per map and side, tribute bag size and shop cadence. *Not cloned:* micro (fighting, kiting, retreat timing). The mimic fights with our engine. Group geometry comes from `data/map_geometry.json`.
`harness/params.py`	Registry of θ: blocks W (18 log-scale multipliers), TARGET (10), FIGHT (9), HOT (1, frozen), U (60, utility arbiter), X (4 extras). Bounds and the initial ES step σ_0 per parameter. Search space $\leftrightarrow$ bot space (`to_search`, `to_bot`).
`harness/pack.py`	Writes `pack.bin` (magic `BCPK`). Per map fingerprint (`data/fingerprints.json`): θ, hotspots, opening, draft, build sections. The bot reads it at runtime (`bots/main/src/pack`).
`tools/bctools/runner.py`	`run_pairs`: many (A,B) pairs over maps in one `cambc run-batch`; `run_matches`: one pairing, both sides. Results carry winner, gems, deaths, replay path.
`scripts/train.ps1`, `train.sh`	Start/stop the daemon in the background; pass-through subcommands.

2.3 Shared state and who writes it

Every file the PCs share travels through git. Each file has one writer, or a semantic merge, so teammates never resolve conflicts by hand.

File	Writer	Content
`league/champion.json`	any PC	`maps`: search-space θ per map; `history`: one entry per promotion (version, host, time, map, gain, h2h, loss id); `bench`: benchmark per version (overall, per team, per map, team×map grid); `tries`: ES cycles per map per host.
`bots/main/src/pack/pack.bin`	any PC	Always regenerated from `champion.json`; never merged.
`league/ledger.json`	lead	`top`: rank/rating snapshots; `pool`: active mimics; `ladder`: our games; `losses`: queue (open / fixed / needs-human); `submissions`: uploads and reverts.
`data/mimic/*.json`	lead	Mimic clone (`<slug>.json`) and pack spec (`<slug>.spec.json`).
`league/review/*.md`	lead + humans	Generated report; the *Reviewer notes* section belongs to humans.
`runs/league/`	local only	`local.json` (ES step σ per map, PFSP win rates, try counts, timers), `league.log`, `lock`, `cycles/NN/` (replays of the last 20 cycles).

Push races. `.gitattributes` marks `league/*.json` and `pack.bin` as `-merge`, so git always stops and `resolve_conflicts` merges them. For `champion.json` it does a 3-way merge per map: if only one side changed a map's θ, that side wins. If both promoted the same map, a head-to-head on that map decides (`h2h_decider`). History is unioned and renumbered by time, so version numbers stay unique. Try counts take

the per-host maximum. Ledger entries are unioned. Any other conflict (code) aborts the rebase; the commit stays local and the log says *merge needs a human*.

Mimic bots `bots/mimic-<slug>` are built locally from `bots/main` plus the spec, and rebuilt only when the spec or the engine code changes (`.league-stamp`). Training *never* modifies a mimic. That keeps "win rate vs Nyx" meaning "vs our best model of Nyx".

3 Decision rules

Fitness of bot b over its games G on the cycle's map:

$$F(b) = \overline{\text{win}} + 0.25\,\overline{\frac{g_{\text{us}} - g_{\text{them}}}{g_{\text{us}} + g_{\text{them}}}} - 0.01\,\overline{\text{deaths}}$$

Matches are deterministic, so the champion and the candidates are scored on identical games.

ES. $\theta' = \text{clip}(\theta \pm \sigma \odot \varepsilon)$ with $\varepsilon \sim \mathcal{N}(0, I)$, 4 pairs. σ per map grows by $\times 1.15$ after a cycle that beat the champion and shrinks by $\times 0.85$ otherwise, clamped to $[0.25\,\sigma_0, 2\,\sigma_0]$. Frozen parameters ($\sigma_0 = 0$) never move.

PFSP opponents. Mimic o is drawn with weight $(1 - p_o)^2 + 0.05$, where p_o is an exponential average of the champion's score against it. Teams that beat us get most of the compute.

Accept gate (all must hold): $F(\text{best}) \geq F(\text{champion}) + 0.04$; a head-to-head against the champion on that map scores ≥ 0.5; on 3 fresh opponents the candidate's fitness is $\geq$ the champion's; suspended (out-of-fuel) turns are $\leq \max(2 \times \text{champion}, 20)$; and the release build compiles.

Release (lead with `--auto-submit`): submit the current champion v if $v >$ the live version, at least 6 h have passed since the last upload, *and* either a ladder loss happened since that upload or the benchmark rose by ≥ 0.05 (overall win rate against the mimics). The upload snapshots `bots/lg-v<N>`, bundles `submission/lg-v<N>.zip`, and runs `cambc submit`.

Revert: once both the live and the previous version have ≥ 10 ranked games and the live one's win rate is > 0.10 lower, the previous version is re-uploaded. This happens at most once per upload.

Escalation: a loss whose map gets 6 ES cycles (summed over all PCs) without a promotion becomes `needs-human`, and training stops chasing it.

4 Human review: epistemic errors

The mimics are models of other teams, and our θ is a model of good play. Both can be wrong in ways ES cannot see. The review report (`league/review/<date>.md`) shows where:

- **Top-10 table, column gap** = (our local win rate vs the team's mimic) − (our real ladder win rate vs the team). A large positive gap (≥ 0.3 with ≥ 3 ladder games) means the mimic is easier than the real team. Its macro was cloned, but its micro was not. Training against it gives false confidence. *Action:* watch their replays, then improve `mimic.py` or write a hand exploiter that copies the missing behaviour.
- **Low-data flag:** a mimic built from fewer than 5 games; its numbers are noise.
- **Losses ES could not fix**, each with a hint:
  - *mimic can't express it*: we beat their mimic on that map locally, but lost the real game. The opponent does something the mimic does not (same action as a large gap).
  - *model can't express it*: we still lose to the mimic after 6 cycles. No setting of θ fixes it, so the engine needs a new feature or knob. That is feature engineering in `crates/` on a `feat/*` branch: gate it, merge it into `league`, and the daemons adopt it.

  Each row lists deaths (us/them), our death contexts, and the `cambc matches download` command for the replay.

- **Reviewer notes**: write findings there. The harness keeps that section when it regenerates the file, and carries it over to the next day's file.

5 Commands

5.1 Set up a PC

```
git clone -b league https://github.com/Ahmed-Abdul-Rahim/battlecode-2026a.git
cd battlecode-2026a
powershell -ExecutionPolicy Bypass -File scripts\setup.ps1     # Linux/macOS: bash scripts/setup.sh
.\.venv\Scripts\Activate.ps1
cambc login                        # lead PC only (platform reads); workers don't need it
python harness\params.py --check  # registry matches learned.rs
pytest -q; cargo test --workspace
```

Run the daemon in a clone you don't edit by hand: it rebases and commits there. Develop in a second clone or a `git worktree`.

5.2 Run the league

```
scripts\train.ps1 start                          # worker
scripts\train.ps1 start --lead                   # lead: + ladder sync, watch, review (no uploads)
scripts\train.ps1 start --lead --auto-submit     # lead that also uploads (ONE PC in the team)
scripts\train.ps1 status                         # champion, benchmark, top 10, losses
scripts\train.ps1 logs                           # follow runs\league\league.log
scripts\train.ps1 stop
scripts\train.ps1 once --no-push                 # one cycle in the foreground, no git (smoke test)
scripts\train.ps1 sync [--no-push]               # lead: ladder + mimics + review now
scripts\train.ps1 bench                          # champion vs every mimic, all maps
scripts\train.ps1 review                         # regenerate today's review file
scripts\train.ps1 submit [--version N]           # manual upload (team decision)
```

On Linux and macOS use `scripts/train.sh` with the same subcommands. `python harness/league.py <cmd>` also works. Set `LEAGUE_HOST` to name a PC in the history (default: its hostname).

5.3 Look at games

```
bctools run main mimic-nyx --maps lanes          # targeted local matches (both sides)
bctools gate main                                # pass/fail vs bots/baseline, Wilson interval
bctools summary runs\league\cycles\03\es\<match>.repl26a
bctools timeline <match>.repl26a renders\t.png   # per-round timeline
bctools render <match>.repl26a 300 renders\r.png # frame at round 300
cambc view <match>.repl26a                       # official visualiser
cambc matches download <matchId> -o runs\review\x.repl26a   # a ladder game from the review
python harness\ladder_study.py runs\ladder_top   # per-team tempo/builds table
```

5.4 Rebuild this document

```
cd docs; latexmk -pdf league.tex; latexmk -c     # MiKTeX or TeX Live
```

6 Debugging

Symptom	What to do
`already running (pid N)`	Another daemon holds `runs/league/lock`. `train stop`; if the pid is dead the lock is ignored anyway.
`cambc ... failed` / HTTP 429	Retried with backoff up to 5 times. Persistent: `cambc login` expired, or the API is down. Training continues offline.
`learned.rs and params.py disagree`	Someone changed the θ layout. Update `harness/params.py` (`--check`) in the same commit.
`build of ... failed`	A broken `bots/main` or `crates/` change was pulled. `bctools sync`; `cambc build bots/main --release`; fix on a branch.
`merge needs a human: [...]`	A code conflict between your local commits and `origin/league`. Resolve by hand, then restart.
`push failed 3 times`	Offline or no permission. The commit stays local and goes out with the next push.
`no mimics yet`	The lead hasn't synced, or `data/mimic/` isn't pulled. Training runs against the anchors meanwhile.
`sync: no games for X`	The team has no downloadable finished games yet; no mimic is built.
Loss on an unknown map	`cambc maps pull`, regenerate `data/fingerprints.json` and `data/map_geometry.json` (§6).
Wrong behaviour after a pull	`git log league -- league/champion.json` shows who promoted what. Reset a map by deleting its key in `maps` and pushing.

Resetting local state is always safe: delete `runs/league/` (local σ, PFSP rates, logs) and the scratch bots `bots/_lg-*`, `bots/mimic-*`. Never hand-edit `league/*.json` while a daemon is running.

New maps need two cached tables. `data/fingerprints.json` maps map name → `world::fingerprint`; the bot prints it in its debug `M` dump. `data/map_geometry.json` holds group centres and spawns. Delete it and run `mimic.map_geometry()` with debug replays of every map in `runs/speedrun/` to rebuild it.

Extending. New trainable parameter: `params.py` + `learned.rs` (see `harness/README.md`). New sparring bot: add it under `bots/` and to `ANCHORS` in `league.py`. Tuning knobs (gain threshold, λ, sync interval, release gap) are constants at the top of `league.py`.